

**Project Interim Report
on
File Compression Simulator
(Using Core Java & OOP Concepts)
RU-100-01-00012
Object oriented Programming with Java**



RUNGTA INTERNATIONAL SKILLS UNIVERSITY

SESSION: 2025-26

Subject teacher name
(Shivansh Mehta)

Submitted By :-
Student name:- Anuj Kumar Singh
Ref No :- 2025-02139
ERP ID :- RU-25-10250

Submitted To :

**RUNGTA INTERNATIONAL SKILLS
UNIVERSITY ,CG
SCHOOL OF COMPUTER SCIENCE ENGINEERING**

INDEX

S.NO	TITLE	PAGE NUMBER
1)	Abstract	1
2)	Introduction and Objectives	2
3)	Scope	3
4)	System Requirements	4
5)	Methodology	5
6)	FlowChart	6-7
7)	Class Diagram	8-9
8)	Implementations	10

INDEX

S.NO	TITLE	PAGE NUMBER
9)	Grantt chart	11
10)	Conclusion	12
11)	References	13

Employee Payroll Management System

Object-Oriented Programming Project Report

Programming Language: Java

Subject: Object-Oriented Programming

Academic Year: 2025–2026

Abstract

The Employee Payroll Management System is a Java-based application developed using core Object-Oriented Programming (OOP) principles. It is designed to automate the management and calculation of employee salaries within an organization, replacing error-prone manual processes with a structured, reusable, and scalable software solution.

The system is needed because organizations employ people of different types — full-time and part-time — each requiring different salary computation logic. Manually handling these calculations is tedious and inconsistent. A software-based system built on OOP concepts provides a clean, extensible solution that mirrors real-world business logic.

The system defines a parent Employee class containing common attributes shared by all employees. It then creates two specialized subclasses — FullTimeEmployee and PartTimeEmployee — each overriding the salary calculation method with their own logic. This design demonstrates inheritance, polymorphism, constructor chaining using the super keyword, and method overriding in a practical, real-world context.

Introduction

Payroll management is a critical function in every organization. Ensuring accurate and fair salary computation for employees with different employment terms is a challenge when performed manually. Errors in payroll can lead to employee dissatisfaction, legal complications, and financial discrepancies. A software-based payroll system provides consistency, accuracy, and ease of management.

This project presents an Employee Payroll Management System implemented in Java using Object-Oriented Programming concepts. The system models different employee types through class hierarchies and automates salary computation based on employment type.

Objectives

- Manage employee records including name, employee ID, and salary details in a structured class hierarchy
- Compute salaries automatically based on employment type — full-time employees receive base salary plus bonus and allowances, while part-time employees are paid based on hours worked and hourly rate
- Reduce manual calculation errors by encapsulating salary logic inside dedicated class methods
- Apply and demonstrate core OOP concepts: inheritance, polymorphism, method overriding, and constructor chaining using the super keyword

Scope

The current version of the Employee Payroll Management System supports two types of employees: FullTimeEmployee and PartTimeEmployee. It is suitable for small organizations or academic demonstrations of OOP design. The system handles the creation of employee objects, computation of their salaries, and display of detailed payroll information.

The project can be extended in the future to include:

- Database integration using MySQL or SQLite for persistent storage of employee records
- Additional employee types such as Contractual, Intern, or Senior Management with their own salary structures
- Tax deduction and net salary computation modules including PF and ESI
- A graphical user interface (GUI) using JavaFX or Swing for improved usability
- Export of payroll reports to PDF or Excel format for official records

System Requirements

Hardware Requirements

- Processor: Intel Core i5 or equivalent (1 GHz or higher)
- RAM: Minimum 4 GB
- Storage: At least 500 MB free disk space
- Display: Any standard monitor with 1024×768 resolution or higher

Software Requirements

- Operating System: Windows 10 / macOS / Linux (Ubuntu 18.04 or higher)
- Programming Language: Java (JDK 8 or higher)
- IDE: VS Code, Eclipse, IntelliJ IDEA, or EditPlus
- Java Runtime Environment (JRE) for executing compiled programs

Methodology

The project follows a structured, step-by-step Object-Oriented design approach. The development process is divided into the following phases:

Step 1 — Define the Parent Employee Class

A parent abstract class `Employee` is created containing attributes common to all employees: `name`, `id`, and `baseSalary`. An abstract method `calculateSalary()` is declared inside this class, which must be implemented by every subclass. A `displayDetails()` method is also defined to print employee information.

Step 2 — Create Subclasses

Two subclasses are created that extend the `Employee` class:

- `FullTimeEmployee` — adds attributes `bonus` and `allowances` to model a salaried employee
- `PartTimeEmployee` — adds attributes `hoursWorked` and `hourlyRate` to model an hourly-wage employee

Step 3 — Implement Parameterized Constructors

Each subclass implements a parameterized constructor that accepts all required values and calls the parent constructor using the `super()` keyword to initialize the inherited attributes `name`, `id`, and `baseSalary`.

Step 4 — Override the Salary Calculation Method

Each subclass overrides the `calculateSalary()` method with its own logic:

- `FullTimeEmployee`: $\text{Total Salary} = \text{baseSalary} + \text{bonus} + \text{allowances}$
- `PartTimeEmployee`: $\text{Total Salary} = \text{hoursWorked} \times \text{hourlyRate}$

Step 5 — Demonstrate Polymorphism

In the main method, `Employee`-type references are used to hold objects of both subclasses. When `calculateSalary()` is called on each reference, Java dynamically dispatches to the correct overridden method based on the actual object type. This demonstrates runtime polymorphism.

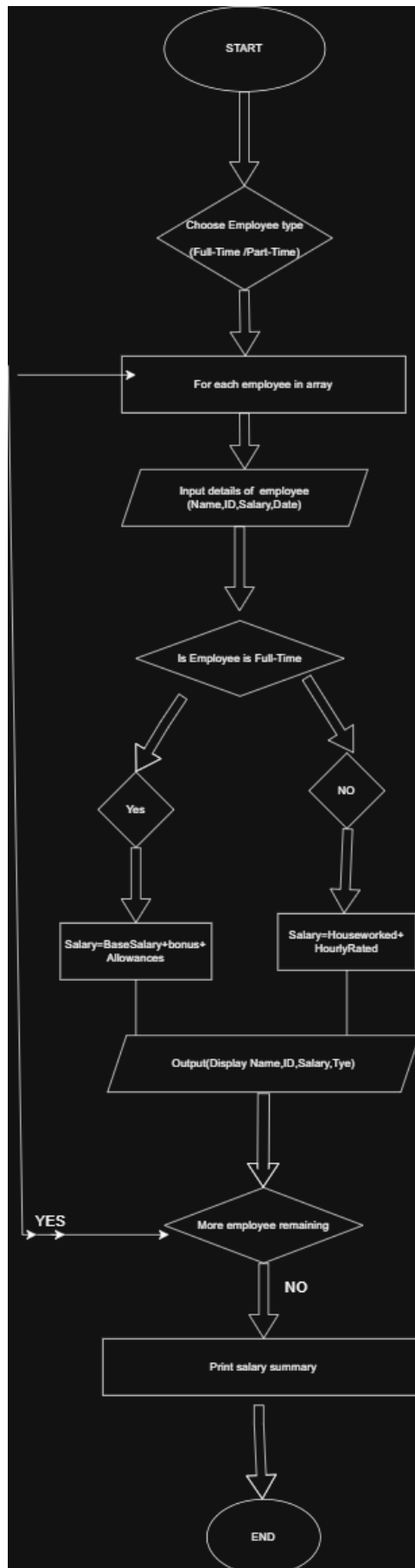
Step 6 — Display Results

Each employee's details — `name`, `ID`, `type`, and computed salary — are printed to the console using the overridden `displayDetails()` method. A final summary table shows all employees and the total payroll expenditure.

Flowchart

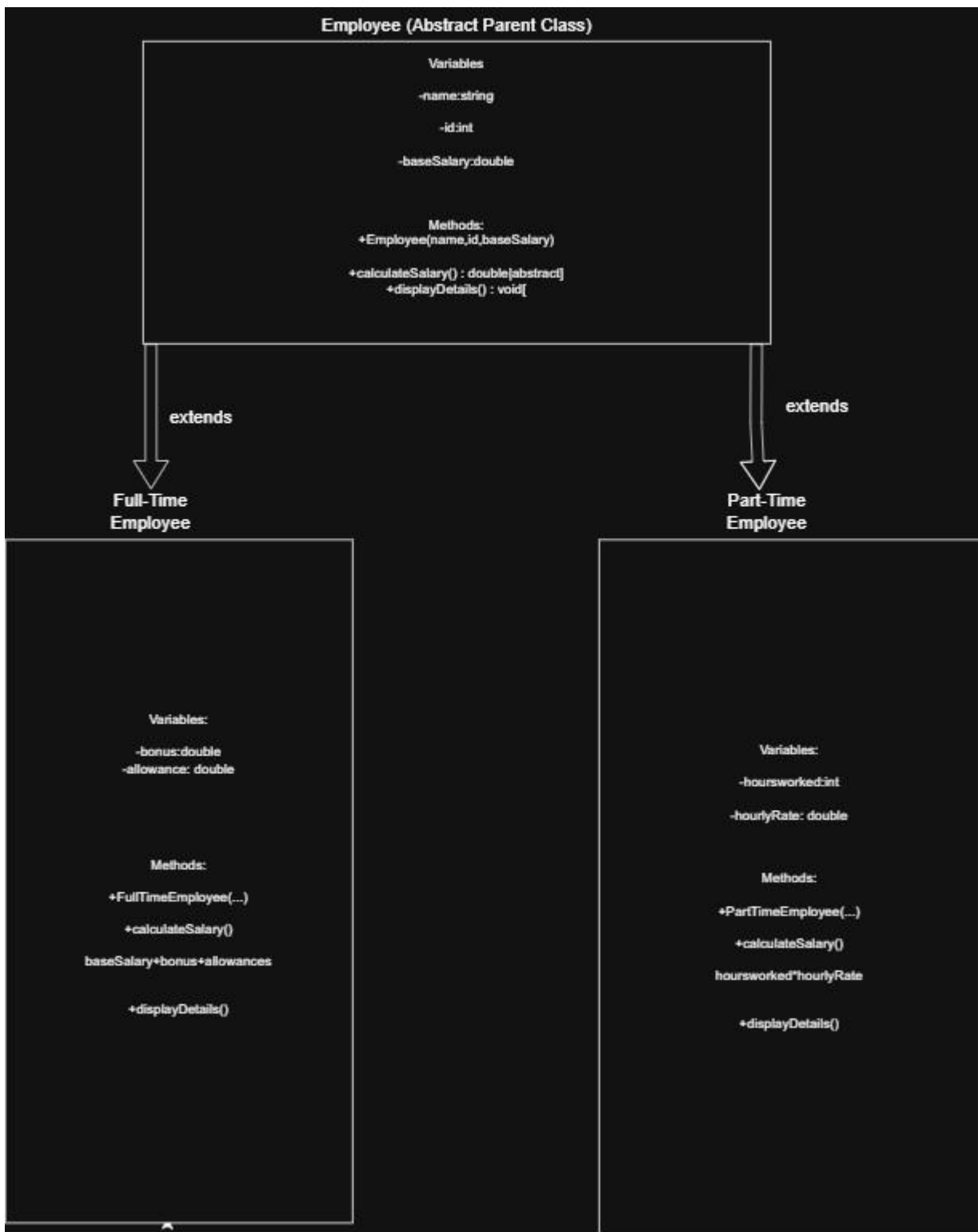
Main Program Flowchart

The following flowchart illustrates the execution flow of the Employee Payroll Management System from program start to salary display and termination.



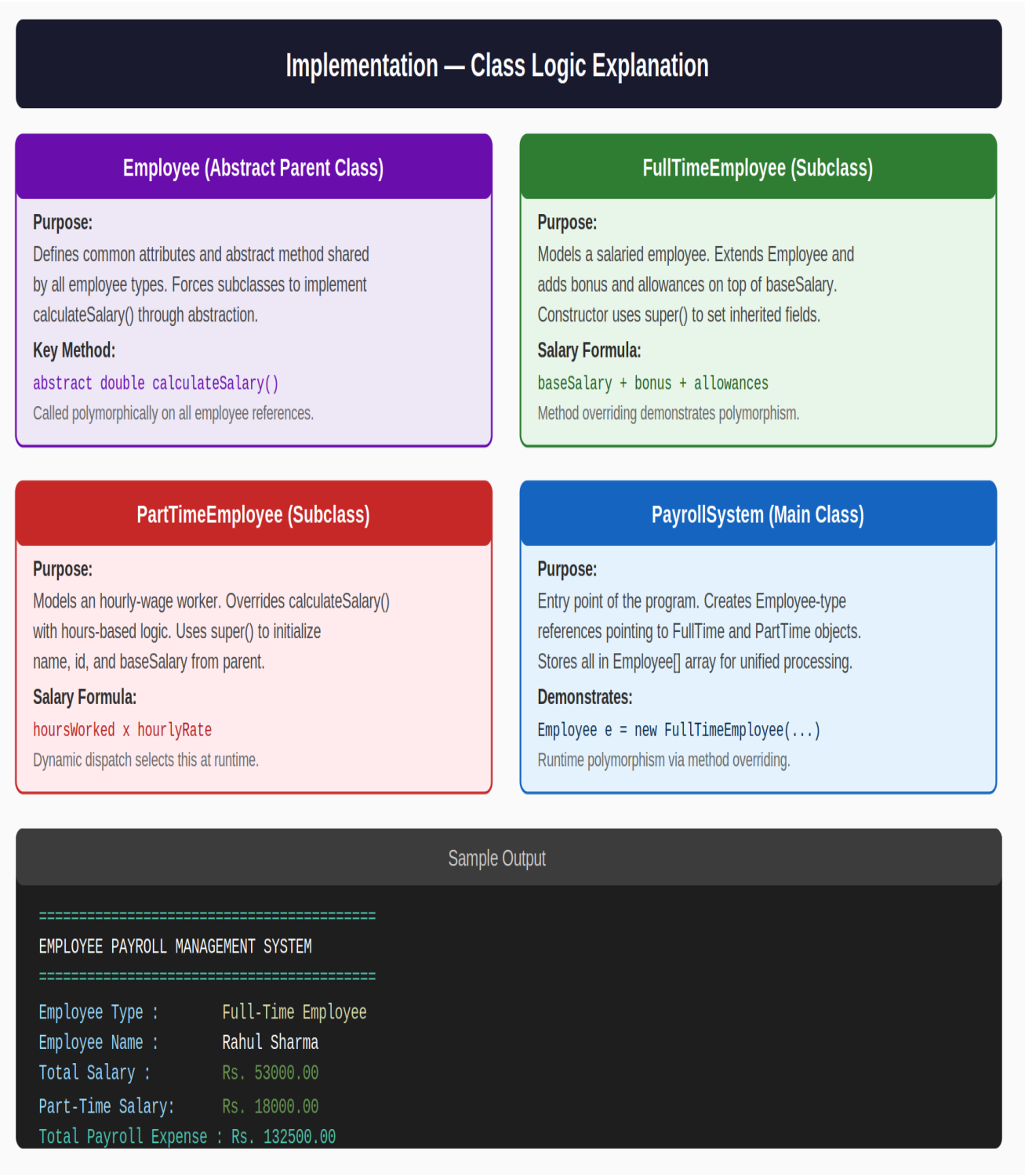
Class Diagram

The following class diagram represents the structure and relationships between the classes in the Employee Payroll Management System.



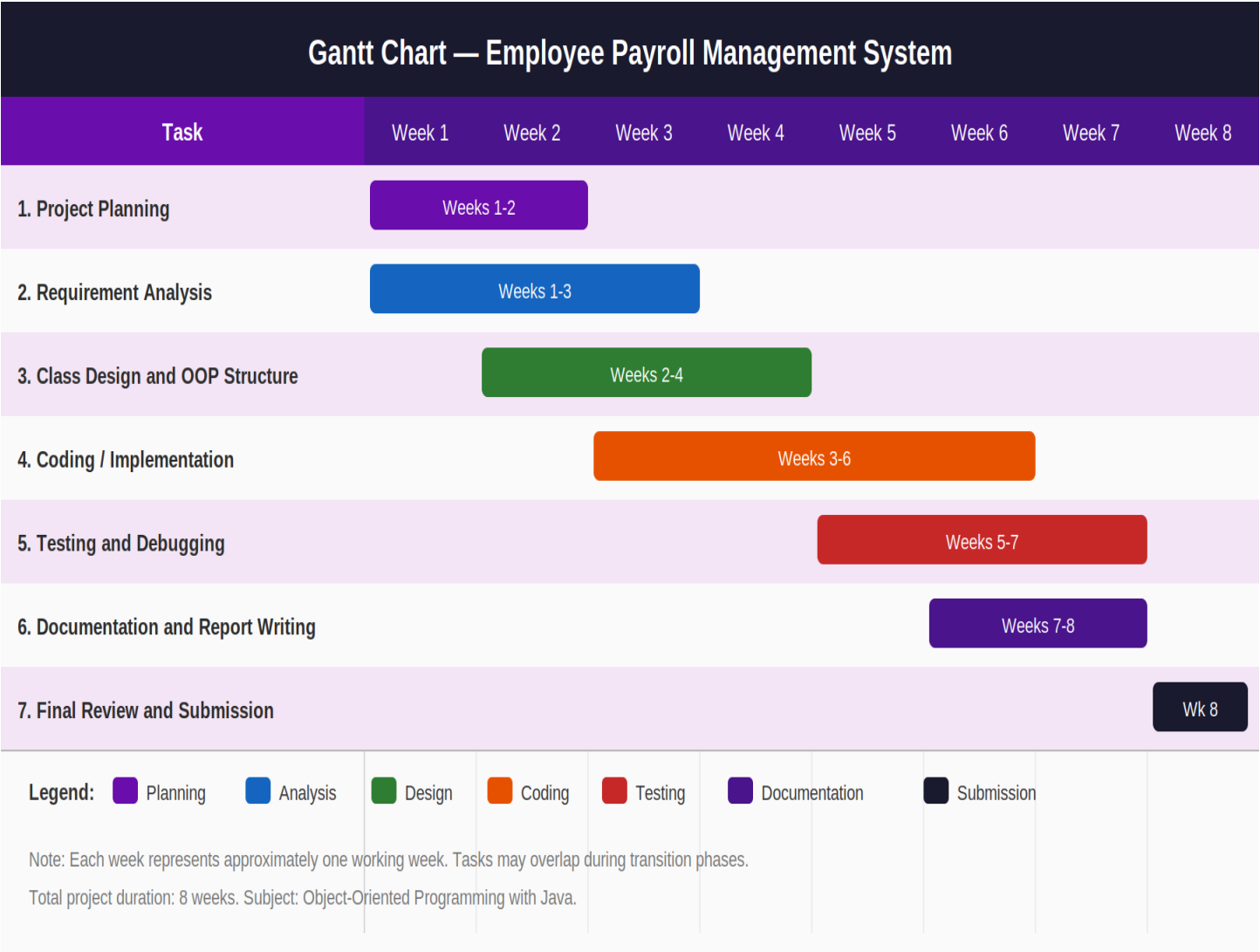
Implementation

The Employee Payroll Management System is implemented using four Java classes, each encapsulating specific responsibilities based on Object-Oriented Programming principles. The diagram below explains the logic of each class, the salary formulas used, and includes a sample terminal output showing the system in action.



Gantt Chart

1. Planning and Structure: The Gantt chart breaks the project into phases and assigns each a timeline across 8 weeks, ensuring all milestones are tracked and completed on schedule.
2. Time Management: Each bar shows when a task starts and ends. Overlapping bars indicate parallel work. This helps avoid delays by distributing work evenly across the project timeline.



Conclusion

The Employee Payroll Management System successfully demonstrates the core principles of Object-Oriented Programming through a practical, real-world application. By designing a structured class hierarchy with a base Employee class and two specialized subclasses — FullTimeEmployee and PartTimeEmployee — the system achieves a clean separation of concerns while remaining flexible and extensible.

The use of inheritance eliminates code duplication by centralizing shared attributes (name, id, baseSalary) in the parent class, while method overriding allows each subclass to define its own salary calculation logic tailored to its employment type. Polymorphism further enhances the design by enabling a single Employee reference to invoke the correct calculateSalary() method at runtime through dynamic dispatch, without the caller needing to know the actual object type.

Constructor chaining via the super() keyword ensures proper and consistent initialization across the hierarchy, reinforcing best practices in object construction. The Main class ties everything together by acting as a driver that creates employee objects and demonstrates how the system behaves with different employee types.

Overall, this project reinforces that good OOP design leads to code that is modular, maintainable, and scalable — new employee types such as ContractEmployee or InternEmployee can be added in the future simply by extending the Employee class, without modifying any existing code. This adherence to the Open/Closed Principle makes the system a strong foundation for a production-grade payroll application.

References

Books:

- [1] H. Schildt, Java: The Complete Reference, 11th ed. New York: McGraw-Hill, 2018.
- [2] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Reading, MA: Addison-Wesley, 1994.

Websites:

- [3] Oracle, "Java Documentation," Oracle. [Online]. Available: <https://docs.oracle.com>
- [4] GeeksforGeeks, "Object-Oriented Programming in Java," GeeksforGeeks. [Online]. Available: <https://www.geeksforgeeks.org/object-oriented-programming-oops-concept-in-java/>

Journal Articles:

- [5] A. Sharma and R. Gupta, "Machine Learning Techniques," International Journal of AI, vol. 5, no. 2, pp. 45-58, 2020.

Conference Papers:

- [6] S. Kumar, "AI in Education," in Proc. IEEE Conf. on AI, 2021, pp. 120-125.
-

PROJECT CODE

```
PayRollSystem.java > PayRollSystem
1 //Anuj kumar singh
2 //erp-10250
3
4 // =====
5 // Employee Payroll System - Single File Version
6 // =====
7
8 abstract class Employee {
9
10     protected String name;
11     protected int id;
12     protected double baseSalary;
13
14     public Employee(String name, int id, double baseSalary) {
15         this.name = name;
16         this.id = id;
17         this.baseSalary = baseSalary;
18     }
19
20     public abstract double calculateSalary();
21
22     public void displayDetails() {
23         System.out.println("-----");
24         System.out.println("Employee Name : " + name);
25         System.out.println("Employee ID : " + id);
26         System.out.printf("Calculated Salary: Rs. %.2f\n", calculateSalary());
27         System.out.println("-----");
28     }
29
30     public String getName() {
31         return name;
32     }
33 }
34
35 // =====
36 // Full-Time Employee Class
37 // =====
38
39 class FullTimeEmployee extends Employee {
40
41     private final double bonus;
42     private double allowances;
43
44     public FullTimeEmployee(String name, int id,
45                             double baseSalary,
46                             double bonus, double allowances) {
47         super(name, id, baseSalary);
48         this.bonus = bonus;
49         this.allowances = allowances;
50     }
51
52     public FullTimeEmployee(double bonus, String name, int id, double baseSalary) {
53         super(name, id, baseSalary);
54         this.bonus = bonus;
55     }
56 }
```

Opening Java Projects: check details

```

6 // =====
7 // Part-Time Employee Class
8 // =====
9
10 class PartTimeEmployee extends Employee {
11
12     private final int hoursWorked;
13     private double hourlyRate;
14
15     @Override
16     public double calculateSalary() {
17         return baseSalary + bonus + allowances;
18     }
19
20     @Override
21     public void displayDetails() {
22         System.out.println("-----");
23         System.out.println("Employee Type : Full-Time Employee");
24         System.out.println("Employee Name : " + name);
25         System.out.println("Employee ID : " + id);
26         System.out.printf("Base Salary : Rs. %.2f\n", baseSalary);
27         System.out.printf("Bonus : Rs. %.2f\n", bonus);
28         System.out.printf("Allowances : Rs. %.2f\n", allowances);
29         System.out.printf("Total Salary : Rs. %.2f\n", calculateSalary());
30         System.out.println("-----");
31     }
32 }
33
34 // =====
35 // Part-Time Employee Class
36 // =====
37
38 class PartTimeEmployee extends Employee {
39
40     private final int hoursWorked;
41     private double hourlyRate;
42
43     @Override
44     public double calculateSalary() {
45         return hoursWorked * hourlyRate;
46     }
47
48     @Override
49     public void displayDetails() {
50         System.out.println("-----");
51         System.out.println("Employee Type : Part-Time Employee");
52         System.out.println("Employee Name : " + name);
53         System.out.println("Employee ID : " + id);
54         System.out.printf("Hours Worked : %d hrs\n", hoursWorked);
55         System.out.printf("Hourly Rate : Rs. %.2f\n", hourlyRate);
56         System.out.printf("Total Salary : Rs. %.2f\n", calculateSalary());
57         System.out.println("-----");
58     }
59 }

```

```

PayRollSystem.java > Language Support for Java(TM) by Red Hat > PayRollSystem
78 // =====
79
80 class PartTimeEmployee extends Employee {
81
82     private final int hoursWorked;
83     private double hourlyRate;
84
85     public PartTimeEmployee(String name, int id,
86                             double baseSalary,
87                             int hoursWorked, double hourlyRate) {
88         super(name, id, baseSalary);
89         this.hoursWorked = hoursWorked;
90         this.hourlyRate = hourlyRate;
91     }
92
93     @Override
94     public double calculateSalary() {
95         return hoursWorked * hourlyRate;
96     }
97
98     @Override
99     public void displayDetails() {
100         System.out.println("-----");
101         System.out.println("Employee Type : Part-Time Employee");
102         System.out.println("Employee Name : " + name);
103         System.out.println("Employee ID : " + id);
104         System.out.printf("Hours Worked : %d hrs\n", hoursWorked);
105         System.out.printf("Hourly Rate : Rs. %.2f\n", hourlyRate);
106         System.out.printf("Total Salary : Rs. %.2f\n", calculateSalary());
107         System.out.println("-----");
108     }
109 }

```

```

        public void displayDetails() {
            System.out.printf("Monthly Rate : Rs. %.2f\n", monthlyRate);
            System.out.printf("Total Salary : Rs. %.2f\n", calculateSalary());
            System.out.println("-----");
        }
    }

// =====
// Main Class (Only public class allowed in one file)
// =====

public class PayRollSystem {

    Run main | Debug main | Run | Debug
    public static void main(String[] args) {

        System.out.println("=====");
        System.out.println("  EMPLOYEE PAYROLL MANAGEMENT SYSTEM  ");
        System.out.println("=====");

        Employee e1 = new FullTimeEmployee("Rahul Sharma", 101, 40000, 8000, 5000);
        Employee e2 = new FullTimeEmployee("Priya Singh", 102, 35000, 6000, 4500);
        Employee e3 = new PartTimeEmployee("Amit Verma", 103, 0, 120, 150);
        Employee e4 = new PartTimeEmployee("Neha Gupta", 104, 0, 80, 200);

        Employee[] employees = {e1, e2, e3, e4};

        System.out.println("\n===== PAYROLL DETAILS =====");
    }
}

```



```

    public static void main(String[] args) {
        for (Employee emp : employees) {
            emp.displayDetails();
        }

        System.out.println("\n===== SALARY SUMMARY =====");

        double totalPayroll = 0;

        for (Employee emp : employees) {
            System.out.printf("%-20s -> Rs. %.2f\n",
                               emp.getName(), emp.calculateSalary());
            totalPayroll += emp.calculateSalary();
        }

        System.out.println("-----");
        System.out.printf("Total Payroll Expense : Rs. %.2f\n", totalPayroll);
        System.out.println("=====");
    }
}

```

OUTPUT

PayRollSystem.java > Language Support for Java(TM) by Red Hat > PayRollSystem

```
152 // -----
153 //      EMPLOYEE PAYROLL MANAGEMENT SYSTEM
154 // =====
155
156 // ===== PAYROLL DETAILS =====
157 // -----
158 // Employee Type   : Full-Time Employee
159 // Employee Name    : Rahul Sharma
160 // Employee ID      : 101
161 // Base Salary      : Rs. 40000.00
162 // Bonus            : Rs. 8000.00
163 // Allowances       : Rs. 5000.00
164 // Total Salary     : Rs. 53000.00
165 // -----
166 // -----
167 // Employee Type   : Full-Time Employee
168 // Employee Name    : Priya Singh
169 // Employee ID      : 102
170 // Base Salary      : Rs. 35000.00
171 // Bonus            : Rs. 6000.00
172 // Allowances       : Rs. 4500.00
173 // Total Salary     : Rs. 45500.00
174 // -----
175 // -----
176 // Employee Type   : Part-Time Employee
177 // Employee Name    : Amit Verma
178 // Employee ID      : 103
179 // Hours Worked     : 120 hrs
180 // Hourly Rate       : Rs. 150.00
181 // Total Salary     : Rs. 18000.00
182 // -----
183 // -----
184 // Employee Type   : Part-Time Employee
185 // Employee Name    : Neha Gupta
186 // Employee ID      : 104
187 // Hours Worked     : 80 hrs
188 // Hourly Rate       : Rs. 200.00
189 // Total Salary     : Rs. 16000.00
190 // -----
191
192 // ===== SALARY SUMMARY =====
193 // Rahul Sharma      -> Rs. 53000.00
194 // Priya Singh        -> Rs. 45500.00
195 // Amit Verma         -> Rs. 18000.00
196 // Neha Gupta         -> Rs. 16000.00
197 // -----
198 // Total Payroll Expense : Rs. 132500.00
199 // =====
```